

Classical Machine Learning for Computer Vision

K-Means, K-Nearest Neighbours, SVM, Decision Trees, and Bag-of-Visual-Words

Contents

1	Introduction: Classical ML in Computer Vision	3
2	Feature Space and Learning	3
3	K-Means Clustering	3
3.1	Motivation	3
3.2	Objective Function	4
3.3	Why Is the Centroid the Mean?	4
3.4	K-Means Pseudocode	4
3.5	K-Means++ Initialization	5
3.6	Caveats	5
4	K-Nearest Neighbours (KNN)	5
4.1	Basic Idea	5
4.2	Distance-Weighted KNN	5
4.3	KNN Pseudocode	6
4.4	Distance Functions	6
4.4.1	Euclidean / L2	6
4.4.2	Squared Euclidean	6
4.4.3	Manhattan / L1	6
4.4.4	Minkowski	6
4.4.5	Chebyshev	7
4.4.6	Cosine	7
4.4.7	Hamming	7
4.5	Choosing K	7
4.6	KNN in CV	7
5	Support Vector Machines (SVM)	7
5.1	Linear SVM Motivation	7
5.2	Hard-Margin SVM	8
5.3	Soft-Margin SVM	8
5.4	Hinge Loss	8
5.5	SVM Training Pseudocode	9
5.6	SVM Inference	9
6	Kernel SVM	9
6.1	Why Kernels?	9
6.2	Dual and Decision Function	9
6.3	Common Kernels	10
6.4	When to Use a Kernel	10

7	Multiclass and Multilabel SVM	10
7.1	One-vs-Rest	10
7.2	One-vs-One	10
7.3	Multilabel SVM	10
8	Decision Trees	11
8.1	Basic Idea	11
8.2	Feature-Space Interpretation	11
8.3	Gini Impurity	11
8.4	Entropy	11
8.5	Information Gain	11
8.6	Decision Tree Training Pseudocode	12
8.7	Continuous versus Discrete Features	12
8.8	When Used in CV	12
8.9	Caveats	12
9	Bag-of-Visual-Words (BoVW)	13
9.1	Motivation	13
9.2	Step 1: Detect and Describe Local Features	13
9.3	Step 2: Learn the Visual Dictionary	13
9.4	Step 3: Vector Quantization	13
9.5	Step 4: Build the Histogram	13
9.6	Step 5: Normalize	14
9.7	Complete BoVW Training Pipeline	14
9.8	Complete BoVW Inference Pipeline	14
9.9	BoVW Pseudocode	14
9.10	Roles of K-Means and SVM	14
9.11	Loss of Spatial Information	15
9.12	Spatial Pyramid Extension	15
10	Comparison of Classical ML Classifiers	15
11	Feature-Space Requirements and Practical Selection	16
12	Training versus Inference Cost	16
13	End-to-End Classical CV Example	16
14	Common Failure Modes	17
15	Summary and Revision	17

1 Introduction: Classical ML in Computer Vision

A classical computer vision recognition pipeline is

Image $\rightarrow$ Feature Detection $\rightarrow$ Feature Description $\rightarrow$ Feature Vector $\rightarrow$ Machine Learning $\rightarrow$ Prediction.

For example,

Image $\rightarrow$ SIFT $\rightarrow$ local descriptors $\rightarrow$ Bag of Visual Words $\rightarrow$ SVM $\rightarrow$ class.

The key distinction is

Feature extraction determines the representation;
ML determines how classes are separated in that representation.

Classical ML is especially useful when datasets are moderate, hand-crafted features are informative, compute is constrained, interpretability matters, or a strong baseline is needed.

2 Feature Space and Learning

Let each image be represented by

$$\mathbf{x}_i = [x_{i1}, \dots, x_{id}]^T \in \mathbb{R}^d, \quad y_i \in \{1, \dots, C\}.$$

The dataset is

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N.$$

The learning problem is to estimate

$$f : \mathbb{R}^d \rightarrow \{1, \dots, C\}.$$

Different algorithms impose different assumptions:

Method	Learning principle	Interpretation of feature space
K-Means	Unsupervised clustering	Points should form compact groups around centroids
KNN	Local similarity	Nearby samples tend to have similar labels
SVM	Margin maximization	Classes can be separated by a large-margin boundary
Decision Tree	Recursive partitioning	Feature thresholds divide the space into useful regions

3 K-Means Clustering

3.1 Motivation

K-Means is an unsupervised algorithm. Given vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$, it partitions them into K clusters. In computer vision its most important classical use is construction of a visual dictionary for Bag-of-Visual-Words (BoVW).

For pooled SIFT descriptors

$$\mathbf{d}_i \in \mathbb{R}^{128},$$

K-Means learns K representative vectors

$$\mathcal{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_K\}.$$

These become visual words.

3.2 Objective Function

Let $z_i \in \{1, \dots, K\}$ be the cluster assignment. K-Means minimizes

$$J = \sum_{i=1}^N \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|_2^2$$

or equivalently

$$J = \sum_{k=1}^K \sum_{i:z_i=k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2.$$

The assignment step is

$$z_i = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$$

and the update step is

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i:z_i=k} \mathbf{x}_i.$$

3.3 Why Is the Centroid the Mean?

For one cluster,

$$J_k = \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2.$$

Differentiating,

$$\frac{\partial J_k}{\partial \boldsymbol{\mu}_k} = -2 \sum_{i \in C_k} (\mathbf{x}_i - \boldsymbol{\mu}_k).$$

Setting this to zero gives

$$\sum_{i \in C_k} \mathbf{x}_i - |C_k| \boldsymbol{\mu}_k = 0,$$

hence

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i.$$

3.4 K-Means Pseudocode

```
INPUT:
  Data X = {x1,...,xN}
  Number of clusters K

1. Initialize K centroids mu1,...,muK.

2. REPEAT:
  Assignment:
  For each xi:
    zi = argmin_k ||xi - muk||^2

  Update:
  For each k:
    muk = mean of points assigned to k

3. UNTIL assignments stop changing,
   centroid movement is small,
```

or maximum iterations is reached.

OUTPUT:

Centroids and cluster assignments

3.5 K-Means++ Initialization

Random initialization can lead to poor local minima. K-Means++ chooses the first centre randomly and subsequent centres with probability proportional to squared distance from the nearest existing centre:

$$P(\mathbf{x}_i) = \frac{D(\mathbf{x}_i)^2}{\sum_j D(\mathbf{x}_j)^2}, \quad D(\mathbf{x}_i) = \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|.$$

3.6 Caveats

- K must be selected.
- The objective is non-convex; initialization matters.
- Euclidean geometry favours compact, roughly spherical clusters.
- Feature scaling can change the result substantially.
- Outliers can move centroids.
- Large K increases dictionary size and quantization cost.

4 K-Nearest Neighbours (KNN)

4.1 Basic Idea

KNN is a supervised, instance-based, non-parametric classifier. Its assumption is

nearby feature vectors tend to have similar labels.

It does not learn an explicit parametric boundary; training mainly consists of storing examples.

For test vector $\mathbf{x}$, calculate

$$d_i = D(\mathbf{x}, \mathbf{x}_i),$$

sort the distances, and select the K smallest. Majority voting gives

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_K(\mathbf{x})} \mathbf{1}[y_i = c].$$

4.2 Distance-Weighted KNN

Closer neighbours can receive larger weights, e.g.

$$w_i = \frac{1}{d_i + \epsilon},$$

with prediction

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_K(\mathbf{x})} w_i \mathbf{1}[y_i = c].$$

4.3 KNN Pseudocode

```
INPUT:
  Training samples {(xi, yi)}
  Test sample x
  K
  Distance D

1. For every training sample xi:
    di = D(x, xi)

2. Sort samples by di.

3. Select the K nearest samples.

4. Perform majority voting or weighted voting.

5. Return predicted class.
```

4.4 Distance Functions

4.4.1 Euclidean / L2

$$D_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^d (x_j - y_j)^2}.$$

Pros: intuitive, rotationally invariant, strong baseline for continuous features. Cons: sensitive to scale and high-dimensional distance concentration.

4.4.2 Squared Euclidean

$$D_2^2(\mathbf{x}, \mathbf{y}) = \sum_j (x_j - y_j)^2.$$

It has exactly the same nearest-neighbour ordering as L2 and avoids the square root.

4.4.3 Manhattan / L1

$$D_1(\mathbf{x}, \mathbf{y}) = \sum_j |x_j - y_j|.$$

Pros: less dominated by individual large coordinate differences and useful for some sparse representations. Cons: still scale-sensitive and task-dependent.

4.4.4 Minkowski

$$D_p(\mathbf{x}, \mathbf{y}) = \left(\sum_j |x_j - y_j|^p \right)^{1/p}.$$

$p = 1$ gives L1; $p = 2$ gives L2. Pros: flexible. Cons: p becomes another design choice.

4.4.5 Chebyshev

$$D_{\infty}(\mathbf{x}, \mathbf{y}) = \max_j |x_j - y_j|.$$

Useful when the largest coordinate deviation is the relevant quantity, but uncommon for standard CV descriptors.

4.4.6 Cosine

$$\text{sim}_{\cos}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2},$$

$$D_{\cos} = 1 - \text{sim}_{\cos}.$$

Pros: compares direction rather than magnitude; useful for normalized histograms. Cons: magnitude information is discarded and zero vectors are problematic.

4.4.7 Hamming

For binary vectors,

$$D_H(\mathbf{x}, \mathbf{y}) = \text{popcount}(\mathbf{x} \oplus \mathbf{y}).$$

Pros: extremely fast and natural for BRIEF/ORB/BRISK. Cons: inappropriate for ordinary real-valued SIFT descriptors.

Distance	Best suited for	Pros	Cons
L2	Dense continuous features	Simple; strong baseline	Scale and dimensionality sensitive
L1	Continuous/sparse features	Less affected by large coordinate errors	Scale sensitive
Minkowski	General numeric features	Flexible through p	Requires selecting p
Cosine	Normalized vectors/histograms	Focuses on direction	Ignores magnitude
Hamming	Binary descriptors	Very fast; hardware friendly	Binary only
Chebyshev	Worst-coordinate error	Simple	Rare in CV descriptors

4.5 Choosing K

Small K gives flexible local decisions but is sensitive to noise. Large K smooths the boundary but can blur minority/local structure. Select K using validation or cross-validation.

4.6 KNN in CV

KNN is attractive for small/medium datasets and when local similarity is meaningful. It becomes expensive when the number of stored examples or descriptor dimensionality is large. This is the curse of dimensionality: as dimension increases, distances can become less discriminative.

5 Support Vector Machines (SVM)

5.1 Linear SVM Motivation

For binary labels $y_i \in \{-1, +1\}$, a linear decision boundary is

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

Prediction is

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b).$$

The vector $\mathbf{w}$ is normal to the hyperplane. The signed distance is

$$\frac{\mathbf{w}^T \mathbf{x} + b}{\|\mathbf{w}\|_2}.$$

Canonical margin boundaries are

$$\mathbf{w}^T \mathbf{x} + b = +1, \quad \mathbf{w}^T \mathbf{x} + b = -1,$$

whose separation is

$$\frac{2}{\|\mathbf{w}\|_2}.$$

Thus maximizing margin is equivalent to minimizing $\frac{1}{2}\|\mathbf{w}\|^2$.

5.2 Hard-Margin SVM

For perfectly separable data,

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1.$$

The optimization is

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to the above constraints. Points on the margin are support vectors.

5.3 Soft-Margin SVM

Real CV data are noisy, so introduce $\xi_i \geq 0$:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i.$$

Then

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i.$$

C controls the trade-off between margin size and training violations. Large C penalizes violations strongly; small C permits more violations.

5.4 Hinge Loss

$$L_i = \max(0, 1 - y_i f(\mathbf{x}_i)).$$

Thus SVM can be viewed as regularized empirical risk minimization:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)).$$

5.5 SVM Training Pseudocode

```
INPUT:
  Training data {(xi, yi)}, yi in {-1,+1}
  Regularization parameter C

1. Form the soft-margin SVM optimization:
   minimize 1/2 ||w||2 + C sumi xi
   subject to yi(wT xi + b) >= 1 - xi
             xi >= 0

2. Solve the convex optimization problem
   using an SVM solver (e.g. SMO/QP methods).

3. Identify support vectors / learned coefficients.

4. Store the resulting model.

OUTPUT:
  Trained SVM model
```

5.6 SVM Inference

For a linear SVM,

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b.$$

The sign gives the binary class. Inference is therefore a dot product plus a bias, making linear SVM attractive for low-latency CV classification.

6 Kernel SVM

6.1 Why Kernels?

A linear boundary may be inadequate in the original feature space. Suppose classes form concentric rings. A straight line cannot separate them. Map

$$\mathbf{x} \rightarrow \phi(\mathbf{x})$$

into a higher-dimensional feature space and separate them linearly there. The kernel trick avoids explicitly constructing ϕ :

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T \phi(\mathbf{z}).$$

6.2 Dual and Decision Function

The dual form is

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

subject to

$$0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0.$$

The kernel SVM decision function is

$$f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b.$$

Only support vectors contribute.

6.3 Common Kernels

Linear:

$$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}.$$

Fast and scalable; excellent for high-dimensional BoVW/HOG-like features.

Polynomial:

$$K(\mathbf{x}, \mathbf{z}) = (\gamma \mathbf{x}^T \mathbf{z} + r)^q.$$

Models feature interactions but introduces more hyperparameters and cost.

RBF/Gaussian:

$$K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|_2^2).$$

Useful for nonlinear boundaries. Large γ makes influence highly local; small γ gives smoother boundaries.

6.4 When to Use a Kernel

Use a nonlinear kernel when a linear classifier is inadequate and the dataset is not too large. Prefer linear SVM when dimensionality is high, the representation is already discriminative, the dataset is large, or inference latency is important. A nonlinear kernel is not automatically superior: it can increase training/inference cost and overfit.

7 Multiclass and Multilabel SVM

7.1 One-vs-Rest

For C classes, train C classifiers. Classifier c learns class c versus all others. For a test vector,

$$\hat{y} = \arg \max_c f_c(\mathbf{x}).$$

Number of classifiers: C . It is simple and efficient, but the “rest” class may be heterogeneous and imbalanced.

7.2 One-vs-One

Train one classifier for every pair of classes:

$$N_{SVM} = \frac{C(C-1)}{2}.$$

Each classifier votes for one class. This gives focused binary problems but quadratic growth in the number of classifiers.

7.3 Multilabel SVM

Multiclass means one class per sample. Multilabel means a sample can have several labels, e.g.

$$\{\text{car, road, building}\}.$$

A simple approach is one binary SVM per label:

$$f_1(\mathbf{x}), \dots, f_L(\mathbf{x}).$$

Predict label ℓ when

$$f_\ell(\mathbf{x}) > \tau_\ell.$$

This Binary Relevance approach is simple but does not explicitly model label correlations. Thresholds can be selected per label rather than forcing the same threshold for all labels.

8 Decision Trees

8.1 Basic Idea

A decision tree recursively partitions feature space. For a continuous feature x_j , a split can be

$$x_j < t.$$

The two regions are

$$\text{left} : x_j < t, \quad \text{right} : x_j \geq t.$$

Leaves store class statistics, usually predicting the dominant class.

8.2 Feature-Space Interpretation

For two features, successive rules such as

$$x_1 < 2.5, \quad x_2 < 1.7$$

create axis-aligned rectangular regions. Thus standard trees naturally implement axis-aligned partitions. Oblique boundaries may require many nodes.

8.3 Gini Impurity

For class proportions $p_1, \dots, p_C$,

$$G = 1 - \sum_{c=1}^C p_c^2.$$

A pure node has $G = 0$.

8.4 Entropy

$$H = - \sum_{c=1}^C p_c \log_2 p_c.$$

Again, a pure node has zero impurity.

8.5 Information Gain

For parent P and children C_j ,

$$I_{\text{after}} = \sum_j \frac{|C_j|}{|P|} I(C_j),$$

so

$$IG = I(P) - I_{\text{after}}.$$

Choose the split producing the largest impurity reduction.

8.6 Decision Tree Training Pseudocode

```
FUNCTION BuildTree(S, Features, depth):  
  
1. If stopping condition holds:  
    create leaf with majority class  
    return leaf  
  
2. best_gain = 0  
   best_feature = None  
   best_threshold = None  
  
3. For each candidate feature fj:  
    For each candidate threshold t:  
        Split S into S_left and S_right  
        Compute weighted impurity  
        gain = impurity(S) - weighted_child_impurity  
        Keep best split.  
  
4. If no useful split exists:  
    create leaf  
    return leaf  
  
5. Split S using the best feature/threshold.  
  
6. Recursively build left and right subtrees.  
  
7. Return the decision node.
```

8.7 Continuous versus Discrete Features

For a continuous feature $x_j \in \mathbb{R}$, candidate thresholds can be placed between sorted values:

$$x_j < t.$$

For a categorical feature,

$$x_j \in \{c_1, c_2, \dots\}$$

splits can separate categories or category subsets, depending on the implementation. Therefore, standard decision trees do not inherently require continuous features.

8.8 When Used in CV

Useful features include color statistics, texture, HOG, BoVW histograms, geometric measurements, and engineered sensor attributes. Trees are attractive when interpretability, mixed feature types, and nonlinear feature interactions matter. A single tree is often weaker than an ensemble on high-dimensional image descriptors.

8.9 Caveats

- Deep trees can overfit.
- Small data changes can produce very different trees.
- Axis-aligned splits can be inefficient for oblique boundaries.
- High-dimensional continuous features create many candidate splits.

- Regularization includes maximum depth, minimum samples per leaf, minimum impurity decrease, and pruning.

9 Bag-of-Visual-Words (BoVW)

9.1 Motivation

A local-feature image contains a variable number of descriptors:

$$D_I = \{\mathbf{d}_1, \dots, \mathbf{d}_{n_I}\}.$$

Since n_I differs from image to image, a fixed-dimensional classifier input is required. BoVW converts the variable-size descriptor set into a fixed K -dimensional histogram.

The analogy is

document $\rightarrow$ words $\rightarrow$ word histogram

and

$$\boxed{\text{image} \rightarrow \text{local descriptors} \rightarrow \text{visual words} \rightarrow \text{histogram}.}$$

9.2 Step 1: Detect and Describe Local Features

For image I detect keypoints and compute descriptors:

$$D_I = \{\mathbf{d}_1, \dots, \mathbf{d}_n\}.$$

For SIFT, each standard descriptor has 128 dimensions.

9.3 Step 2: Learn the Visual Dictionary

Pool descriptors from training images:

$$\mathcal{D} = \{\mathbf{d}_1, \dots, \mathbf{d}_M\}.$$

Run K-Means with K clusters:

$$\boxed{\mathcal{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_K\}.$$

These centroids are visual words.

9.4 Step 3: Vector Quantization

For descriptor $\mathbf{d}_i$, assign the nearest visual word:

$$\boxed{q(\mathbf{d}_i) = \arg \min_k D(\mathbf{d}_i, \mathbf{v}_k).$$

For Euclidean quantization,

$$q(\mathbf{d}_i) = \arg \min_k \|\mathbf{d}_i - \mathbf{v}_k\|_2.$$

Thus every descriptor becomes an integer in $\{1, \dots, K\}$.

9.5 Step 4: Build the Histogram

The count for visual word k is

$$\boxed{h_k = \sum_{i=1}^n \mathbf{1}[q(\mathbf{d}_i) = k].}$$

The image representation is

$$\boxed{\mathbf{h} = [h_1, \dots, h_K]^T \in \mathbb{R}^K.}$$

9.6 Step 5: Normalize

L1 normalization:

$$\hat{h}_k = \frac{h_k}{\sum_j h_j + \epsilon}.$$

L2 normalization:

$$\hat{\mathbf{h}} = \frac{\mathbf{h}}{\|\mathbf{h}\|_2 + \epsilon}.$$

TF-IDF-style weighting can reduce the importance of visual words appearing in almost every image.

9.7 Complete BoVW Training Pipeline

Training images → keypoints → local descriptors → K-Means
→ visual dictionary → quantization → BoVW histograms → classifier training.

9.8 Complete BoVW Inference Pipeline

Test image → keypoints → descriptors → nearest visual words
→ histogram → normalization → trained classifier → prediction.

9.9 BoVW Pseudocode

```
TRAINING:
1. Detect local features and descriptors in all training images.
2. Pool training descriptors.
3. Run K-Means with vocabulary size K.
4. For every training image:
    initialize h[1..K] = 0
    for each descriptor d:
        k = nearest visual word
        h[k] += 1
    normalize h
5. Train SVM/KNN/Decision Tree using image histograms.
6. Store dictionary and classifier.
```

```
INFERENCE:
1. Detect keypoints in test image.
2. Compute descriptors.
3. Quantize each descriptor to nearest visual word.
4. Build K-dimensional histogram.
5. Normalize histogram.
6. Apply trained classifier.
```

9.10 Roles of K-Means and SVM

The canonical pipeline is

SIFT → K-Means → BoVW → SVM.

Their roles are different:

SIFT	local appearance representation
K-Means	visual vocabulary learning
BoVW	fixed-dimensional image representation
SVM	supervised class decision boundary

9.11 Loss of Spatial Information

Standard BoVW counts visual words but ignores their locations. Two images can therefore have similar histograms while having different spatial arrangements:

BoVW preserves approximately “what”, but loses much of “where”.

9.12 Spatial Pyramid Extension

Divide the image into regions, for example

$$1 \times 1, \quad 2 \times 2, \quad 4 \times 4,$$

compute a histogram in each region, and concatenate them:

$$\mathbf{h}_{SPM} = [\mathbf{h}_{1 \times 1}, \mathbf{h}_{2 \times 2}, \mathbf{h}_{4 \times 4}].$$

This introduces coarse spatial information at the cost of a larger feature vector.

10 Comparison of Classical ML Classifiers

Method	Training idea	Strengths	Weaknesses	Typical CV use
K-Means	Minimize within-cluster distance	Simple; useful for quantization; scalable baseline	Unsupervised; requires K ; initialization/scale sensitive	Visual dictionary; clustering
KNN	Use labels of nearby examples	Very simple; no explicit training; flexible boundaries	Slow inference; memory-heavy; dimensionality sensitive	Small/medium feature datasets
Linear SVM	Maximum-margin hyperplane	Strong high-dimensional baseline; fast inference	Linear boundary only	BoVW, HOG, engineered features
Kernel SVM	Maximum margin in implicit feature space	Nonlinear decision boundaries	Higher cost; kernel/hyperparameter tuning	Moderate-sized nonlinear problems
Decision Tree	Recursive impurity reduction	Interpretable; mixed feature types; nonlinear rules	Overfitting; unstable; axis-aligned partitions	Structured/engineered CV features
Random Forest	Ensemble of randomized trees	More robust than one tree; nonlinear interactions	Larger model; less interpretable	General classical classification

11 Feature-Space Requirements and Practical Selection

Algorithm	Feature type	Scaling sensitivity	Key requirement
K-Means	Numeric vectors	High	Meaningful distance metric
KNN	Numeric/suitable metric representation	High	Meaningful neighbourhood definition
SVM	Numeric vectors	Usually important	Appropriate scaling and regularization
Decision Tree	Numeric and categorical	Low	Useful discriminative splits; implementation-specific categorical support

For distance-based methods, standardization can be used:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}.$$

However, descriptor-specific normalization should be considered; normalization is not a universal preprocessing rule.

12 Training versus Inference Cost

Method	Training	Inference	Main computational issue
K-Means	Moderate/high	Low after dictionary creation	Repeated distance calculations during clustering
KNN	Minimal	High	Distance to stored training samples
Linear SVM	Moderate	Very low	Dot product with $\mathbf{w}$
Kernel SVM	Potentially high	Potentially high	Kernel evaluation against support vectors
Decision Tree	Moderate	Low	Tree traversal
Random Forest	Higher	Moderate	Multiple tree traversals

A useful engineering distinction is

KNN stores examples; SVM stores a learned decision rule.

13 End-to-End Classical CV Example

For object classification:

Training images $\rightarrow$ SIFT $\rightarrow$ descriptors $\rightarrow$ K-Means
 $\rightarrow$ BoVW histograms $\rightarrow$ linear SVM training.

For a new image:

Test image $\rightarrow$ SIFT $\rightarrow$ descriptors $\rightarrow$ visual-word assignment
 $\rightarrow$ histogram $\rightarrow$ SVM $\rightarrow$ class.

The classifier never sees the raw image; it sees a fixed-dimensional engineered representation.

14 Common Failure Modes

1. Poor feature representation: the classifier cannot recover information discarded by the representation.
2. Wrong distance metric: especially damaging for KNN/K-Means.
3. Unscaled features: large numerical ranges dominate distance calculations.
4. Poor visual vocabulary: too small causes quantization loss; too large increases dimensionality and cost.
5. Kernel overfitting: excessive flexibility or unsuitable C, γ can fit training noise.
6. Tree overfitting: very deep trees can memorize samples.
7. KNN inference bottleneck: exhaustive comparison against a large training set is expensive.
8. BoVW spatial information loss: histograms discard precise arrangement.
9. Class imbalance: majority classes can dominate votes, impurity measures, and one-vs-rest training.
10. Data leakage: the visual dictionary must be learned from training data only; test data must not influence K-Means.

15 Summary and Revision

The four core algorithms can be remembered as questions:

K-Means:	Which prototype is this feature closest to?
KNN:	Which labelled examples are closest?
SVM:	Which decision boundary maximizes separation?
Decision Tree:	Which feature threshold best partitions the data?

The canonical classical CV pipeline is

Local descriptors $\rightarrow$ K-Means $\rightarrow$ BoVW $\rightarrow$ classifier.
--

The overall design principle is

Representation $\rightarrow$ distance/similarity $\rightarrow$ learning objective $\rightarrow$ decision rule.
--

Algorithm selection should consider feature representation, dataset size, dimensionality, class structure, latency, memory, and interpretability.